

UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

ACTIVIDAD 9

SW TEST PLANNING

**CARBAJAL JIMENEZ LAURA IXCHEL
221799122**

**SEMINARIO INTEGRACION: DESARROLLO
D01 V0734 265996 CICLO-VE26**



**UNIVERSIDAD DE
GUADALAJARA**
Red Universitaria de Jalisco

UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS



Seminario Integración: Diseño

Documento de planificación de pruebas de software

Mtra. Sonia Jazmín García De Lira

Nombre del proyecto: AQUAMET Home

Propósito del documento

El presente documento define la planificación de pruebas de software del proyecto AQUAMET Home, un sistema IoT gamificado para el monitoreo inteligente de tinacos domésticos. Su objetivo es establecer de forma sistemática y trazable los casos de prueba que permitan verificar y validar que el sistema cumple con los requisitos funcionales y no funcionales definidos en la Especificación de Requisitos de Software (SRS).

El documento se organiza en función de los avances del proyecto, distribuyendo los casos de prueba de acuerdo con la etapa del desarrollo en que cada funcionalidad estará disponible para ser evaluada. Los casos de prueba se clasifican en tres niveles de prioridad y complejidad: HIGH, para funcionalidades críticas cuya falla compromete el funcionamiento central del sistema; MEDIUM, para funcionalidades importantes que afectan la experiencia del usuario; y LOW, para funcionalidades secundarias cuya validación se realiza una vez que el sistema está integrado.

Todas las pruebas se diseñaron tomando como referencia los requisitos documentados en el SRS del proyecto AQUAMET Home y se estructuran de forma que cualquier miembro del equipo o evaluador externo pueda replicarlas bajo las mismas condiciones. El cronograma refleja el porcentaje de cobertura alcanzado en cada avance, garantizando que las pruebas de mayor criticidad se ejecuten en las etapas más tempranas del desarrollo.

Avances del Proyecto

Avance 1: Configuración de hardware y backend base

- Instalación y conexión del sensor ultrasónico al ESP32 mediante GPIO.
- Instalación y conexión del sensor magnético de tapa al ESP32.
- Programación del firmware del ESP32 para leer ambos sensores y formatear los datos en JSON.
- Configuración de la conectividad Wi-Fi en el ESP32 y conexión al broker MQTT.
- Despliegue del broker MQTT en el servidor de backend.
- Creación del endpoint POST /mediciones en la API REST para recibir y almacenar datos.
- Configuración inicial de la base de datos en la nube con la tabla de mediciones.
- Prueba de transmisión básica: ESP32 → MQTT → API → Base de datos.

Avance 2: Integración MQTT y API REST

- Implementación de la lógica de reconexión automática del ESP32 ante pérdida de Wi-Fi.
- Desarrollo de los endpoints GET /nivel/actual y GET /mediciones con filtrado por fechas.
- Implementación del endpoint GET /consumo/resumen para historial diario, semanal y mensual.
- Configuración de índices en la base de datos para optimizar consultas por rango de fechas.
- Pruebas de integración del flujo completo: sensor → ESP32 → broker → API → base de datos.
- Validación de la frecuencia de medición periódica y continuidad durante 10 minutos.

Avance 3: Aplicación móvil Android y sistema de alertas push

- Desarrollo de la pantalla principal (Dashboard) con indicador de nivel y estado de tapa.
- Desarrollo de la pantalla de historial con gráficas de consumo diario, semanal y mensual.
- Integración de la app con la API REST mediante solicitudes HTTP/HTTPS.
- Configuración de OneSignal y registro del dispositivo Android para notificaciones push.

- Implementación de la lógica de alertas en la API para nivel crítico y tapa abierta.
- Pruebas de envío y recepción de notificaciones con app abierta y cerrada.

Avance 4: Módulo de gamificación e integración completa

- Desarrollo del módulo de gamificación en la app: puntaje, insignias y progreso.
- Implementación del endpoint GET /usuario/puntos en la API REST.
- Lógica de otorgamiento de puntos por revisiones, tapa cerrada y reducción de consumo.
- Pruebas de otorgamiento correcto de puntos e insignias según comportamiento del usuario.
- Prueba de estabilidad del sistema completo durante 2 horas continuas.
- Prueba de consistencia del resumen semanal entre la app y la base de datos.
- Corrección de errores detectados en pruebas previas y cierre de deuda técnica.

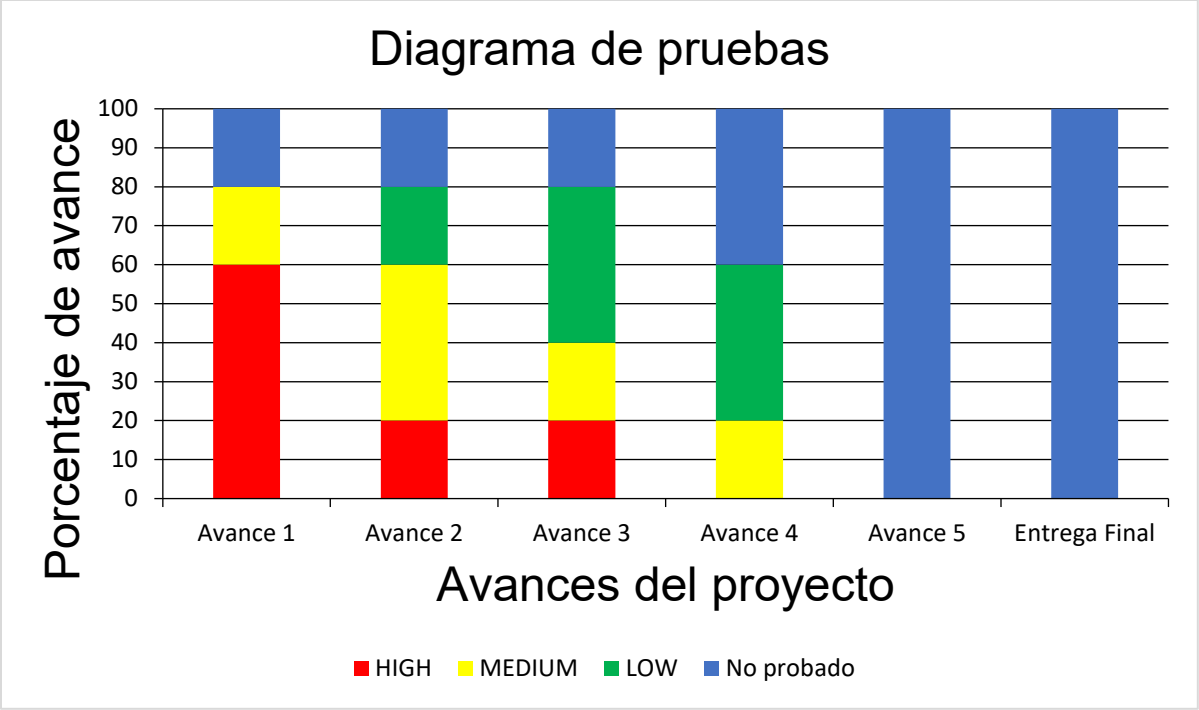
Avance 5: Cierre, correcciones y entrega final

- Corrección de todos los errores detectados en pruebas de avances anteriores.
- Revisión de seguridad: HTTPS, autenticación de usuarios y protección de datos.
- Documentación final: actualización del SRS, SDD y documento de planificación de pruebas.
- Prueba de regresión completa: ejecución de todos los casos de prueba HIGH, MEDIUM y LOW.
- Preparación del entorno de demostración y presentación final del prototipo.

Cronograma de casos de prueba

	Avance 1	Avance 2	Avance 3	Avance 4	Avance 5	Entrega Final
Fase de entrega	Hardware y backend base	MQTT y API REST	App móvil y alertas push	Gamificación e integración	Cierre y correcciones	Entrega final
Duración	Mar–Abr	May–Jun	Jul–Ago	Sep–Oct	Oct–Nov	Nov
Implementación del Software	20%	50%	75%	90%	95%	100%
Casos de prueba HIGH	60%	80%	100%	100%	100%	100%
Casos de prueba MEDIUM	20%	60%	80%	100%	100%	100%
Casos de prueba LOW	0%	20%	60%	100%	100%	100%

El diagrama siguiente refleja el porcentaje de casos de prueba ejecutados por tipo en cada avance. Los casos HIGH se concentran en los primeros avances por ser críticos para el funcionamiento del sistema. Los casos MEDIUM se incorporan progresivamente conforme la app y las alertas están disponibles. Los casos LOW se ejecutan en su mayoría en el Avance 4, una vez que el sistema está completamente integrado.



Casos de Prueba HIGH

TestCaseHIGH_0	Planned?	Test type
Test Name: Medición de nivel de agua con sensor ultrasónico	YES	HIGH
Avance		
Avance 1: Configuración de hardware y backend base		
Test Objective		
Verificar que el sensor ultrasónico mide correctamente la distancia al nivel de agua y el ESP32 la convierte a porcentaje y litros estimados.		
Test Environment description		
Dispositivo Android 8.0+, ESP32 con firmware cargado, red Wi-Fi 2.4 GHz activa, backend desplegado (Python/Node.js), base de datos en la nube operativa, aplicación móvil instalada.		
Requirements covered		
RF-01: Medición del nivel de agua		
Test Functionality		
Paso 1: Encender el ESP32 con el sensor conectado. Paso 2: Colocar agua en el tinaco a un nivel conocido (50%). Paso 3: Leer el valor reportado por el sensor en el monitor serial. Paso 4: Medir manualmente la distancia con cinta métrica. Paso 5: Comparar la medición del sensor con la medición manual.		
Expected Results		
1- El ESP32 enciende sin errores. 2- El tinaco contiene agua al 50% de su capacidad. 3- El sensor reporta un valor de distancia en cm. 4- Se obtiene la medición de referencia manual. 5- La diferencia entre la medición del sensor y la manual no supera 2 cm.		

TestCaseHIGH_1	Planned?	Test type
Test Name: Transmisión de datos al broker MQTT	YES	HIGH
Avance		
Avance 1: Configuración de hardware y backend base		
Test Objective		
Verificar que el ESP32 publica los datos de los sensores en el broker MQTT con el formato JSON correcto y dentro del tiempo esperado.		
Test Environment description		
Dispositivo Android 8.0+, ESP32 con firmware cargado, red Wi-Fi 2.4 GHz activa, backend desplegado (Python/Node.js), base de datos en la nube operativa, aplicación móvil instalada.		

Requirements covered
RF-03: Transmisión de datos al backend
Test Functionality
Paso 1: Conectar el ESP32 a la red Wi-Fi. Paso 2: Suscribirse al topic del broker MQTT desde MQTT Explorer. Paso 3: Esperar a que el ESP32 publique un mensaje. Paso 4: Verificar los campos del JSON recibido. Paso 5: Comprobar el tiempo transcurrido entre la medición y la recepción del mensaje.
Expected Results
1- El ESP32 se conecta a Wi-Fi sin errores. 2- La suscripción al topic queda activa. 3- El mensaje se recibe en el suscriptor. 4- El JSON contiene: nivel_cm, nivel_pct, litros_estimados, tapa_estado, timestamp. 5- El mensaje se recibe en menos de 5 segundos desde la medición.

TestCaseHIGH_2	Planned?	Test type
Test Name: Almacenamiento de medición en base de datos	YES	HIGH
Avance		
Avance 2: Integración MQTT y API REST		
Test Objective		
Verificar que la API REST recibe el mensaje del broker y lo almacena correctamente en la base de datos en la nube.		
Test Environment description		
Dispositivo Android 8.0+, ESP32 con firmware cargado, red Wi-Fi 2.4 GHz activa, backend desplegado (Python/Node.js), base de datos en la nube operativa, aplicación móvil instalada.		
Requirements covered		
RF-04: Almacenamiento del historial de consumo		
Test Functionality		
Paso 1: Publicar un mensaje de prueba con valores conocidos en el broker MQTT. Paso 2: Verificar que la API recibe el mensaje (HTTP 200). Paso 3: Consultar la base de datos directamente para confirmar la inserción. Paso 4: Comparar los valores almacenados con los enviados. Paso 5: Verificar que el timestamp es correcto.		
Expected Results		
1- El mensaje se publica exitosamente. 2- La API responde con HTTP 200. 3- El registro aparece en la tabla de mediciones. 4- Los valores almacenados coinciden exactamente con los enviados. 5- El timestamp del registro difiere en menos de 2 segundos del momento de envío.		

TestCaseHIGH_3	Planned?	Test type
-----------------------	-----------------	------------------

Test Name: Visualización del nivel actual en la app móvil	YES	HIGH
Avance		
Avance 3: Aplicación móvil y alertas push		
Test Objective		
Verificar que la aplicación móvil muestra correctamente el nivel de agua actual y el estado de la tapa obtenidos de la API REST.		
Test Environment description		
Dispositivo Android 8.0+, ESP32 con firmware cargado, red Wi-Fi 2.4 GHz activa, backend desplegado (Python/Node.js), base de datos en la nube operativa, aplicación móvil instalada.		
Requirements covered		
RF-05: Visualización en la aplicación móvil		
Test Functionality		
Paso 1: Abrir la aplicación en el dispositivo Android. Paso 2: Navegar a la pantalla principal (Dashboard). Paso 3: Verificar que el indicador de nivel muestra porcentaje y litros actuales. Paso 4: Modificar físicamente el nivel de agua en el tinaco. Paso 5: Actualizar la app y verificar que el valor cambia.		
Expected Results		
1- La app abre sin errores. 2- El Dashboard es visible en menos de 3 segundos. 3- El indicador muestra porcentaje y litros coherentes con el nivel actual. 4- El nivel físico del tinaco cambia. 5- El nuevo valor se refleja en la app en menos de 10 segundos tras la actualización.		

TestCaseHIGH_4	Planned?	Test type
Test Name: Envío de notificación push por nivel crítico	YES	HIGH
Avance		
Avance 3: Aplicación móvil y alertas push		
Test Objective		
Verificar que el sistema envía una notificación push al usuario cuando el nivel de agua baja del umbral configurado, incluso con la app cerrada.		
Test Environment description		
Dispositivo Android 8.0+, ESP32 con firmware cargado, red Wi-Fi 2.4 GHz activa, backend desplegado (Python/Node.js), base de datos en la nube operativa, aplicación móvil instalada.		
Requirements covered		
RF-06: Notificaciones push por eventos críticos		
Test Functionality		

Paso 1: Configurar el umbral de nivel crítico al 20% en la API. Paso 2: Cerrar la aplicación móvil en el dispositivo. Paso 3: Reducir el nivel de agua del tinaco por debajo del 20%. Paso 4: Esperar a que el ESP32 transmita la medición al backend. Paso 5: Verificar que la notificación push llega al dispositivo Android.

Expected Results

1- El umbral queda configurado correctamente. 2- La app está cerrada en el dispositivo. 3- El nivel físico está por debajo del 20%. 4- El ESP32 transmite la medición y la API detecta la condición. 5- La notificación 'Nivel crítico de agua' aparece en el dispositivo en menos de 30 segundos.

Casos de Prueba MEDIUM

TestCaseMEDIUM_0	Planned?	Test type
Test Name: Detección de tapa abierta con sensor magnético	YES	MEDIUM
Avance		
Avance 1: Configuración de hardware y backend base		
Test Objective		
Verificar que el sensor magnético detecta correctamente cuando la tapa del tinaco es removida y reporta el cambio de estado al ESP32.		
Test Environment description		
Dispositivo Android 8.0+, ESP32 con firmware cargado, red Wi-Fi 2.4 GHz activa, backend desplegado (Python/Node.js), base de datos en la nube operativa, aplicación móvil instalada.		
Requirements covered		
RF-02: Detección del estado de la tapa		
Test Functionality		
Paso 1: Instalar el sensor magnético en el borde del tinaco con la tapa cerrada. Paso 2: Leer el estado desde el monitor serial (debe indicar 'cerrada'). Paso 3: Remover la tapa del tinaco. Paso 4: Leer nuevamente el estado. Paso 5: Volver a colocar la tapa y verificar que el estado regresa a 'cerrada'.		
Expected Results		
1- El sensor queda correctamente instalado. 2- El monitor serial muestra tapa_estado: cerrada. 3- La tapa es removida físicamente. 4- El monitor serial muestra tapa_estado: abierta en menos de 2 segundos. 5- Al colocar la tapa el estado regresa a 'cerrada' en menos de 2 segundos.		

TestCaseMEDIUM_1	Planned?	Test type
Test Name: Reconexión automática del ESP32 tras pérdida de Wi-Fi	YES	MEDIUM
Avance		
Avance 2: Integración MQTT y API REST		
Test Objective		
Verificar que el ESP32 se reconecta automáticamente al broker MQTT tras una interrupción temporal de la red Wi-Fi sin intervención manual.		
Test Environment description		
Dispositivo Android 8.0+, ESP32 con firmware cargado, red Wi-Fi 2.4 GHz activa, backend desplegado (Python/Node.js), base de datos en la nube operativa, aplicación móvil instalada.		

Requirements covered
RF-03: Transmisión de datos al backend
Test Functionality
Paso 1: Verificar que el ESP32 transmite datos normalmente. Paso 2: Desconectar el router Wi-Fi durante 30 segundos. Paso 3: Reconectar el router. Paso 4: Esperar hasta 60 segundos. Paso 5: Verificar que el ESP32 reanuda la publicación de mensajes en el broker.
Expected Results
1- El ESP32 publica datos sin interrupciones. 2- La conexión Wi-Fi se pierde y el ESP32 registra el error en el serial. 3- El router vuelve a estar disponible. 4- El ESP32 detecta la red y reconecta. 5- Los mensajes MQTT se reanudan en menos de 60 segundos sin intervención.

TestCaseMEDIUM_2	Planned?	Test type
Test Name: Historial de consumo diario en la app	YES	MEDIUM
Avance		
Avance 3: Aplicación móvil y alertas push		
Test Objective		
Verificar que la aplicación muestra el historial de consumo diario con datos coherentes obtenidos de la API REST.		
Test Environment description		
Dispositivo Android 8.0+, ESP32 con firmware cargado, red Wi-Fi 2.4 GHz activa, backend desplegado (Python/Node.js), base de datos en la nube operativa, aplicación móvil instalada.		
Requirements covered		
RF-04: Almacenamiento del historial; RF-05: Visualización en app		
Test Functionality		
Paso 1: Asegurar que existen al menos 24 mediciones del día actual en la base de datos. Paso 2: Abrir la app y navegar a la pantalla de historial. Paso 3: Seleccionar la vista de consumo diario. Paso 4: Verificar que la gráfica muestra valores correctos. Paso 5: Comparar los valores de la gráfica con los registros directos de la base de datos.		
Expected Results		
1- Existen registros del día actual en la base de datos. 2- La pantalla de historial carga sin errores. 3- La vista diaria es seleccionable. 4- La gráfica representa los niveles a lo largo del día de forma continua. 5- Los valores de la gráfica coinciden con los registros de la base de datos con tolerancia de $\pm 1\%$.		

TestCaseMEDIUM_3	Planned?	Test type
-------------------------	-----------------	------------------

Test Name: Notificación push por tapa abierta	YES	MEDIUM
Avance		
Avance 3: Aplicación móvil y alertas push		
Test Objective		
Verificar que el sistema envía una notificación push cuando la tapa del tinaco es removida y el backend detecta el evento.		
Test Environment description		
Dispositivo Android 8.0+, ESP32 con firmware cargado, red Wi-Fi 2.4 GHz activa, backend desplegado (Python/Node.js), base de datos en la nube operativa, aplicación móvil instalada.		
Requirements covered		
RF-06: Notificaciones push; RF-02: Detección de tapa		
Test Functionality		
Paso 1: Asegurar que el sistema opera normalmente con la tapa cerrada. Paso 2: Remover la tapa del tinaco. Paso 3: Esperar a que el ESP32 transmita el cambio de estado al backend. Paso 4: Verificar que la API despacha la alerta a OneSignal. Paso 5: Confirmar la recepción de la notificación en el dispositivo Android.		
Expected Results		
1- El sistema opera normalmente con tapa_estado: cerrada. 2- El sensor detecta tapa_estado: abierta. 3- El ESP32 transmite el evento en el siguiente ciclo. 4- La API evalúa la condición y envía solicitud a OneSignal. 5- La notificación 'Tapa del tinaco abierta' aparece en el dispositivo en menos de 30 segundos.		

TestCaseMEDIUM_4	Planned?	Test type
Test Name: Otorgamiento de puntos por reducción de consumo	YES	MEDIUM
Avance		
Avance 4: Módulo de gamificación e integración		
Test Objective		
Verificar que el módulo de gamificación otorga puntos correctamente cuando el usuario reduce su consumo respecto a la semana anterior.		
Test Environment description		
Dispositivo Android 8.0+, ESP32 con firmware cargado, red Wi-Fi 2.4 GHz activa, backend desplegado (Python/Node.js), base de datos en la nube operativa, aplicación móvil instalada.		
Requirements covered		

RF-07: Sistema de gamificación
Test Functionality
Paso 1: Registrar datos de consumo de la semana anterior en la base de datos. Paso 2: Registrar datos de la semana actual con consumo menor. Paso 3: Abrir la app y navegar al módulo de gamificación. Paso 4: Verificar los puntos otorgados por la reducción. Paso 5: Confirmar que el puntaje es consistente con el porcentaje de reducción registrado.
Expected Results
1- Los datos de consumo previo están correctamente registrados. 2- Los datos actuales reflejan un consumo menor. 3- La sección de gamificación carga sin errores. 4- El sistema muestra los puntos otorgados por la reducción. 5- Los puntos reflejan fielmente el porcentaje de ahorro calculado por el backend.

Casos de Prueba LOW

TestCaseLOW_0	Planned?	Test type
Test Name: Actualización periódica del nivel sin interacción del usuario	YES	LOW
Avance		
Avance 2: Integración MQTT y API REST		
Test Objective		
Verificar que el sistema genera y transmite mediciones periódicamente sin necesidad de intervención del usuario durante un período de 10 minutos.		
Test Environment description		
Dispositivo Android 8.0+, ESP32 con firmware cargado, red Wi-Fi 2.4 GHz activa, backend desplegado (Python/Node.js), base de datos en la nube operativa, aplicación móvil instalada.		
Requirements covered		
RF-01: Medición del nivel; RF-03: Transmisión de datos		
Test Functionality		
Paso 1: Encender el sistema y esperar 10 minutos sin interacción. Paso 2: Revisar los registros del backend para contar las mediciones recibidas. Paso 3: Calcular la frecuencia de medición observada. Paso 4: Comparar con el intervalo configurado (60 segundos). Paso 5: Verificar que no existen brechas superiores a 90 segundos entre mediciones.		
Expected Results		
1- El sistema opera sin intervención. 2- Se registran aproximadamente 10 mediciones en 10 minutos. 3- La frecuencia es de ~1 medición por minuto. 4- El intervalo real no supera 1.5x el intervalo configurado. 5- No existen brechas de más de 90 segundos entre registros consecutivos.		

TestCaseLOW_1	Planned?	Test type
Test Name: Visualización del estado de la tapa en el Dashboard	YES	LOW
Avance		
Avance 3: Aplicación móvil y alertas push		
Test Objective		
Verificar que el Dashboard muestra el estado actual de la tapa con el indicador visual de color correcto (verde cerrada, rojo abierta).		
Test Environment description		

Dispositivo Android 8.0+, ESP32 con firmware cargado, red Wi-Fi 2.4 GHz activa, backend desplegado (Python/Node.js), base de datos en la nube operativa, aplicación móvil instalada.
Requirements covered
RF-05: Visualización en la aplicación móvil
Test Functionality
Paso 1: Abrir la app con la tapa del tinaco cerrada. Paso 2: Verificar el indicador de estado de tapa en el Dashboard (verde - 'Cerrada'). Paso 3: Remover la tapa del tinaco. Paso 4: Actualizar la app. Paso 5: Verificar que el indicador cambia a rojo - 'Abierta'.
Expected Results
1- La app carga sin errores. 2- El indicador muestra 'Cerrada' con fondo verde. 3- La tapa es removida físicamente. 4- La app consulta el estado actualizado desde la API. 5- El indicador cambia a 'Abierta' con fondo rojo en menos de 10 segundos.

TestCaseLOW_2	Planned?	Test type
Test Name: Visualización de insignias en el módulo de gamificación	YES	LOW
Avance		
Avance 4: Módulo de gamificación e integración		
Test Objective		
Verificar que las insignias obtenidas por el usuario se muestran correctamente en la sección de gamificación con nombre, descripción y fecha de obtención.		
Test Environment description		
Dispositivo Android 8.0+, ESP32 con firmware cargado, red Wi-Fi 2.4 GHz activa, backend desplegado (Python/Node.js), base de datos en la nube operativa, aplicación móvil instalada.		
Requirements covered		
RF-07: Sistema de gamificación		
Test Functionality		
Paso 1: Asignar manualmente una insignia al usuario en la base de datos. Paso 2: Abrir la app y navegar al módulo de gamificación. Paso 3: Verificar que la insignia aparece en la lista de logros. Paso 4: Verificar que muestra nombre, descripción y fecha. Paso 5: Confirmar que las insignias no obtenidas aparecen como bloqueadas.		
Expected Results		
1- La insignia queda registrada en la base de datos. 2- La sección de gamificación carga sin errores. 3- La insignia asignada es visible en la lista de logros. 4- Se muestran nombre, descripción y fecha de obtención correctamente. 5- Las insignias no obtenidas se muestran con estado bloqueado.		

TestCaseLOW_3	Planned?	Test type
Test Name: Funcionamiento continuo del sistema durante 2 horas	YES	LOW
Avance		
Avance 4: Módulo de gamificación e integración		
Test Objective		
Verificar que el sistema completo opera de forma estable durante 2 horas continuas sin interrupciones críticas ni pérdida de datos.		
Test Environment description		
Dispositivo Android 8.0+, ESP32 con firmware cargado, red Wi-Fi 2.4 GHz activa, backend desplegado (Python/Node.js), base de datos en la nube operativa, aplicación móvil instalada.		
Requirements covered		
RF-03: Transmisión de datos; RF-05: Visualización en app		
Test Functionality		
Paso 1: Encender el sistema completo (ESP32, backend, app). Paso 2: Dejar el sistema operar 120 minutos sin intervención. Paso 3: Al finalizar, revisar los registros del backend para verificar continuidad. Paso 4: Verificar que la app sigue mostrando datos actualizados. Paso 5: Revisar logs de errores del ESP32 y backend.		
Expected Results		
1- El sistema completo está operativo. 2- El sistema opera 120 minutos sin intervención. 3- Los registros muestran mediciones continuas sin brechas superiores a 90 segundos. 4- La app muestra el nivel actualizado correctamente. 5- No se registran errores críticos en los logs durante el período de prueba.		

TestCaseLOW_4	Planned?	Test type
Test Name: Consistencia del resumen de consumo semanal	YES	LOW
Avance		
Avance 4: Módulo de gamificación e integración		
Test Objective		
Verificar que el resumen de consumo semanal mostrado en la app es consistente con los datos almacenados en la base de datos.		
Test Environment description		
Dispositivo Android 8.0+, ESP32 con firmware cargado, red Wi-Fi 2.4 GHz activa, backend desplegado (Python/Node.js), base de datos en la nube operativa, aplicación móvil instalada.		
Requirements covered		

RF-04: Almacenamiento del historial; RF-05: Visualización en app

Test Functionality

Paso 1: Asegurar que existen 7 días completos de mediciones en la base de datos. Paso 2: Calcular manualmente el consumo promedio de la semana. Paso 3: Abrir la app y navegar a la vista de historial semanal. Paso 4: Comparar el consumo mostrado en la app con el calculado manualmente. Paso 5: Verificar que los totales diarios individuales también coinciden.

Expected Results

1- Existen registros de los últimos 7 días en la base de datos. 2- Se obtiene el consumo promedio de referencia. 3- La vista semanal carga sin errores. 4- El consumo promedio de la app difiere en menos del 5% del calculado manualmente. 5- Los totales diarios son coherentes con los registros individuales de mediciones.

Referencias

Documento de Especificación de Requisitos del Software (SRS) — AQUAMET Home, Versión 1.0, Junio 2026.

Documento de Diseño de Software (SDD) — AQUAMET Home, Versión 1.0, Junio 2026.

Jan, F., Min-Allah, N., Saeed, S., Iqbal, S. Z., & Ahmed, R. (2022). IoT-Based Solutions to Monitor Water Level, Leakage, and Motor Control for Smart Water Tanks. *Water*, 14(3), 309. <https://doi.org/10.3390/w14030309>

Raza, A., & Ahmed, M. S. (2024). Smart water management systems using IoT technologies. *JSPGA*. <https://www.jspga.com/index.php/jspga/article/view/68>